

NumPy: Numerical Python

Why learn Numpy?

In today's data-driven world, everything can be quantified and represented as numbers—from user behavior on websites to patterns in medical data. Understanding these numerical representations is essential for machine learning, data analysis, and scientific computing. NumPy serves as the foundational package for numerical operations in Python, making it easier for you to delve into these domains.

Numpy Arrays

This tutorial introduces you to NumPy's data containers, offering a quick look at how to manage data sets in Python. In the realm of mathematics, **vectors** and **matrices** are fundamental units represented as sequences and grids of numbers, respectively. They are integral to linear algebra, a crucial area in computational tasks underlying modern Machine Learning and Artificial Intelligence. In NumPy, these structures are simplified as one-dimensional (1D) and two-dimensional (2D) arrays, and the library even supports higher-dimensional arrays with ease.

To get started, import NumPy like this:

```
In [10]: import numpy as np
```

A 1D arrays can be created from a list with the NumPy function `array`. If the items of the list have different type, they are converted to a common type when creating the array. A simple example follows.

```
In [11]: mylist = [2, 7, 14, 5, 9]
mylist
```

```
Out[11]: [2, 7, 14, 5, 9]
```

```
In [12]: type(mylist)
```

```
Out[12]: list
```

```
In [13]: arr1 = np.array([1,2,3,4])
arr1
```

```
Out[13]: array([1, 2, 3, 4])
```

This looks the same, but it's a very different beast! The constraint that everything has the same type is a useful one as it allows us to operate on the array more naturally. Try these for comparison:

```
In [14]: print(arr1 + 1)
print(mylist + [1])
print(arr1 * 2)
print(mylist * 2)
```

```
[2 3 4 5]
[2, 7, 14, 5, 9, 1]
[2 4 6 8]
[2, 7, 14, 5, 9, 2, 7, 14, 5, 9]
```

In case of mixed types (never do this!) you will get unexpected results:

```
In [15]: arr_mixed = np.array([1, 'a', 3, 4])
arr_mixed
```

```
Out[15]: array(['1', 'a', '3', '4'], dtype='<U21')
```

There are two types involved with arrays. The type of the array is `array`, its elements also have a type which can be checked using `myarray.dtype`:

```
In [16]: type(arr_mixed)
```

```
Out[16]: numpy.ndarray
```

```
In [17]: arr_mixed.dtype
```

```
Out[17]: dtype('<U21')
```

A 2D array can be directly created from a list of lists of equal length. The terms are entered row-by-row:

```
In [18]: my_list_of_lists = [  
          [0, 7, 2, 3],  
          [3, 9, -5, 1]  
        ]  
my_list_of_lists
```

```
Out[18]: [[0, 7, 2, 3], [3, 9, -5, 1]]
```

```
In [19]: arr2 = np.array(my_list_of_lists)  
arr2
```

```
Out[19]: array([[ 0,  7,  2,  3],  
                [ 3,  9, -5,  1]])
```

Although we visualize a vector as a column (or as a row) and a matrix as a rectangular arrangement, with rows and columns, it is not so in the computer. The 1d array is just a sequence of elements of the same type, neither horizontal nor vertical. It has one **axis**, which is the 0-axis.

In a similar way, a 2d array is a sequence of 1d arrays of the same length and type. It has two axes. When we visualize it as rows and columns, `axis=0` means *across rows*, while `axis=1` means *across columns*.

The number of terms stored along an axis is the **dimension** of that axis. The dimensions are collected in the attribute `shape` :

```
In [20]: arr1.shape
```

```
Out[20]: (4,)
```

```
In [21]: arr2.shape
```

```
Out[21]: (2, 4)
```

```
In [22]: arr3 = np.random.randn(2,3,4)  
arr3.shape
```

```
Out[22]: (2, 3, 4)
```

This is a tuple, meaning that you can extract one of the elements, but you cannot reassign it.

```
In [23]: print(arr2.shape[0])
```

```
2
```

```
In [24]: arr2.shape[0] = 123
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[24], line 1  
----> 1 arr2.shape[0] = 123  
TypeError: 'tuple' object does not support item assignment
```

You try it:

1. Create a prediction function `price` that predicts the price of an apartment based on its surface:

Price = 100,000 + 5,000 x (surf in m2)

2. Calculate the price of a house of 130 m2.
3. Calculate the price over a range of surfaces 50,60,...,130

```
In [25]: ...
```

```
Out[25]: Ellipsis
```

NumPy functions

NumPy incorporates vectorized forms of the **mathematical functions** of the package `math`. A **vectorized function** is one that, when applied to an array, returns an array with same shape, whose terms are the values of the function on the corresponding terms of the original array. For instance, the NumPy function `np.max` takes the maximum along an axis of a numeric array:

```
In [33]: # Heights in cm [Female, Male]
heights = np.array([[160, 175],
                    [155, 180],
                    [165, 170],
                    [162, 178],
                    [158, 172]])

heights.shape
```

```
Out[33]: (5, 2)
```

```
In [39]: np.mean(heights, axis=0)
```

```
Out[39]: array([160., 175.])
```

```
In [ ]: heights.shape
```

You can also tell `numpy` to do this calculation along the rows or columns using the `axis` parameter. Let's try calculating the mean for Females v.s. Males using the `np.mean()` function.

```
In [ ]: np.max(heights,axis=0)
```

```
Out[ ]: array([165, 180])
```

DEMO: Play around with this simulation to get a feeling for how it works.



NumPy also provides common mathematical and statistical functions, such as `median`, `max`, `sum`, `sqrt`, `std`, `quantile` etc.

Functions that are defined in terms of vectorized functions are automatically vectorized. Let's try this with an exercise:

You try it:

Below is a 2D array with measurements for 10 people. Each row is one person; the two columns are height (cm) and weight (kg).

1. Write a function `bmi(h, w)` that computes BMI from height in cm and weight in kg.
2. Use it to compute the BMI for all 10 people at once.
3. What is the average height / weight / BMI?

$$\text{BMI} = \frac{\text{weight}}{(\text{height in meters})^2}$$

```
In [ ]: # Each row: [height_cm, weight_kg]
people = np.array([ [160, 55],
                    [175, 70],
                    [155, 52],
                    [180, 77],
                    [165, 58],
                    [170, 68],
                    [162, 54],
                    [178, 75],
                    [158, 53],
                    [172, 72]])

def calc_bmi(...):
    return ...

calc_bmi(people[:,0], people[:,1])
```

Subsetting arrays

Slicing a 1D array is done the same as for a list:

```
In [ ]: arr1 = np.array([5,4,2,41])
```

```
In [68]: arr1[:3]
```

```
Out[68]: array([5, 4, 2])
```

The same applies to two-dimensional arrays, but we need two indexes within the square brackets. The first index selects the rows (`axis=0`), and the second index the columns (`axis=1`):

```
In [69]: people[[1,2], :]
```

```
Out[69]: array([[175, 70],
               [155, 52]])
```

Step intervals can be used if we need to select only every n-th element of the array:

```
In [70]: arr1, arr1[::2]
```

```
Out[70]: (array([ 5,  4,  2, 41]), array([5, 2]))
```

One special case of this which is used quite often is to use steps of -1, which is equivalent to reversing the array.

```
In [71]: arr1, arr1[::-1]
```

```
Out[71]: (array([ 5,  4,  2, 41]), array([41,  2,  4,  5]))
```

```
In [72]: arr1
```

```
Out[72]: array([ 5,  4,  2, 41])
```

```
In [73]: arr1[::-2]
```

```
Out[73]: array([5, 2])
```

DEMO: You can test your indexing knowledge here:



Filtering Subsets of an array can also be extracted by means of expressions which acts as filters. Any expression involving an array is evaluated in Python as a Boolean array (called a **Boolean mask**):

```
In [1]: heights = people[:,0]
weights = people[:,1]

f_tall = heights > 170

f_tall
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[1], line 1
----> 1 heights = people[:,0]
      2 weights = people[:,1]
      4 f_tall = heights > 170

NameError: name 'people' is not defined
```

We can use this as an index, and it will remove anyone in the dataset who is too tall:

```
In [ ]: heights[f_tall]
```

These can be combined into complex expressions as suited for the application:

```
In [ ]: filter_tall = heights > 160
filter_short = heights < 180

heights[filter_tall & filter_short ]
```

Since both heights and weights are of the same size, we can filter either on the other:

```
In [ ]: weights[filter_tall]
```

Extra exercises for home

You try it

What is the average BMI of people taller than 170cm?

```
In [ ]:
```

You try it

Let's perform RFM (Recency, Frequency, Monetary) analysis - a key technique in customer segmentation and marketing analytics.

RFM analysis is a behavior-based technique used to segment customers by examining their transaction history. It evaluates customers based on three dimensions:

- **Recency (R):** How recently a customer has made a purchase.
- **Frequency (F):** How often a customer makes a purchase.
- **Monetary Value (M):** How much money a customer spends on purchases.

The idea of this battle-tested technique is that it allows us to (a) categorize customers into distinct groups for targeted marketing (b) find out recent / frequent buyers which are more likely to respond to new offers (c) focus efforts on segments that are more likely to generate higher returns.

Let's start with generating the dataset (you don't need to worry about this code for now).

```
In [104...] import numpy as np

def generate_rfm_aggregate_data(num_customers=100, max_days_since_last_purchase=365, max_frequency=25, max_monetary=10000):
    """
    Generate synthetic aggregate RFM data for customers.

    Parameters:
    - num_customers: int, number of unique customers.
    - max_days_since_last_purchase: int, maximum number of days since the last purchase.
    - max_frequency: int, maximum number of transactions made by a customer.
    - max_monetary: int, maximum total amount spent by a customer.

    Returns:
    - data: np.ndarray, 2D array with columns: CustomerID, Recency, Frequency, Monetary.
    """
    np.random.seed(42) # For reproducibility

    # I use random numbers here for the sake of example, this is an artificial dataset
    recency = np.round(np.random.randint(1, max_days_since_last_purchase + 1, num_customers), 2)
    frequency = np.round(np.random.randint(1, max_frequency + 1, num_customers), 2)
    monetary = np.round(np.random.uniform(10.0, max_monetary, num_customers), 2)

    # Combining all three to output 1 matrix
    data = np.column_stack((recency, frequency, monetary))

    return data

rfm_data = generate_rfm_aggregate_data(num_customers=10000)

rfm_data
```

```
Out[104...] array([[1.03000e+02, 1.80000e+01, 7.29780e+03],
       [3.49000e+02, 1.20000e+01, 2.07865e+03],
       [2.71000e+02, 1.60000e+01, 3.77658e+03],
       ...,
       [3.64000e+02, 9.00000e+00, 4.75868e+03],
       [1.40000e+02, 1.30000e+01, 7.37955e+03],
       [1.90000e+02, 3.00000e+00, 2.01410e+02]])
```

You try it

Create a function `rfm_score` that computes a simple RFM score:

$$\text{Score} = (\text{max_recency} - \text{recency}) + \text{frequency} + (\text{monetary} / 1000)$$

Higher is better.

```
In [ ]: ## RFM
rfm_data = generate_rfm_aggregate_data(num_customers=1000)

def rfm_score(data):

    return ...

# 2. Compute RFM scores for all customers.
scores = ...
```

```
In [ ]: idx = np.argmax(scores)

print("Most valuable customer is ....",idx)
print("With a score of", scores[idx])
```

Most valuable customer is 895
With a score of 394.0

```
In [ ]: # 3. Find top 5 customers with the highest scores.
top5_idx = np.argsort(scores)

sorted = np.argsort(scores)
top5_idx = sorted[-5:][::-1]
top5_scores = scores[top5_idx]
print("Top 5 customers (indices):", top5_idx)
print("Top 5 scores:", top5_scores)
```

SOLUTIONS

```
In [ ]: def calc_price(surf):
        return 100000 + 50000 * surf

## Price of a house of 130 m2
calc_price(130)

## Price of a range of surfaces
surfaces = np.array(list(range(50, 131)))
prices = calc_price(surfaces)
```

```
In [ ]: # Each row: [height_cm, weight_kg]
people = np.array([ [160, 55],
                    [175, 70],
                    [155, 52],
                    [180, 77],
                    [165, 58],
                    [170, 68],
                    [162, 54],
                    [178, 75],
                    [158, 53],
                    [172, 72]])

heights = people[:, 0]
weights = people[:, 1]

def bmi(h, w):
    return w / (h / 100.)**2

all_bmis = bmi(heights, weights)
np.mean(all_bmis)
```

```
In [ ]: filter_tall = heights > 170
np.mean(bmi(heights[filter_tall], weights[filter_tall]))
```

```
In [ ]: ## RFM
rfm_data = generate_rfm_aggregate_data(num_customers=1000)

def rfm_score(data):
    score = (np.max(data[:, 0]) - data[:, 0]) + data[:, 1] + (data[:, 2] / 1000)
    return np.round(score)

# 2. Compute RFM scores for all customers.
scores = rfm_score(rfm_data)
```

```
# 3. Find top 5 customers with the highest scores.  
top5_idx = np.argsort(scores)  
  
sorted = np.argsort(scores)  
top5_idx = sorted[-5:][::-1]  
top5_scores = scores[top5_idx]  
print("Top 5 customers (indices):", top5_idx)  
print("Top 5 scores:", top5_scores)
```